Accelerating Polynomial System Solving via factorization and UNSAT Core Learning

Ilay Menahem, Michael Shalyt, Ido Kaminer

August 6, 2026

Abstract

Groebner basis methods are a powerful tool for in algebraic geometry and polynomial ideal theory. However, they can be computationally expensive, especially for large systems of polynomials. In this paper, we...

1 Introduction

2 Preliminaries and Notation

2.1 Notation

Let K be a field, let $\overline{K}$ be an algebraic closure of K, and let $K[x_1, \ldots, x_n]$ be the polynomial ring in n variables over K. We write the input system as

$$P = \{p_1, \ldots, p_m\} \subseteq K[x_1, \ldots, x_n].$$

We assume that each p_i is nonzero and fix a factorization

$$p_i = a_i \prod_{f \in C_i} f^{e_{i,f}},$$

where $a_i \in K \setminus \{0\}$, C_i is the set of distinct nonconstant factors appearing in the factorization, and $e_{i,f} \geq 1$ is the multiplicity of f.

For any set of polynomials $Q \subseteq K[x_1, \ldots, x_n]$, we write $\langle Q \rangle$ for the ideal generated by Q and define its zero set over the algebraic closure by

$$V(Q) = \{x \in \overline{K}^n : q(x) = 0 \text{ for every } q \in Q\}.$$

For $X \subseteq \overline{K}^n$, its vanishing ideal is

$$I(X) = \{g \in K[x_1, \ldots, x_n] : g(x) = 0 \text{ for every } x \in X\}.$$

Unless stated otherwise, satisfiability refers to solutions over $\overline{K}$. Hilbert's Nullstellensatz [1] gives

$$V(Q) = \emptyset \quad \iff \quad \langle Q \rangle = \langle 1 \rangle.$$

Equivalently, the reduced Gröbner basis of $\langle Q \rangle$ is $\{1\}$. A set $U \subseteq Q$ is an unsatisfiable core of Q if $V(U) = \emptyset$.

A hitting set of $C_1, \ldots, C_m$ is a set $H \subseteq \bigcup_{i=1}^{m} C_i$ such that $H \cap C_i \neq \emptyset$ for every i. It is minimal if none of its proper subsets is also a hitting set. We denote the collection of all minimal hitting sets of $C_1, \ldots, C_m$ by $\mathcal{H}$.

2.2 Polynomial ideals

2.3 Basic algebraic geometry

3 Method

Let us say we have a very large polynomial system P, we factorize the polynomials in P and get a set of factors $C_1, \ldots, C_m$. We iterate over all the minimal hitting sets of $C_1, \ldots, C_m$, and solve the polynomial system corresponding to each minimal hitting set. We show that the union of the solution sets our algorithm returns is the full solution set of P. Additionally, we optimize the order of iteration over the minimal hitting sets, and learn unsatisfiable cores to skip hitting sets that contain them and thus unsolvable.

Algorithm 1 Branching

Require: A set of polynomials $P = \{p_1, \ldots, p_m\}$.

Interreduce the polynomials ▷ Optional
Initialize $C_1, \ldots, C_m$ as the sets of factors of $p_1, \ldots, p_m$
Initialize $C \leftarrow \emptyset$ an unsatisfiable cores set
Initialize $S \leftarrow \emptyset$ a solutions set
for Each minimal hitting set HS of $C_1, \ldots, C_m$ **do**
    if HS contains an unsatisfiable core from C **then**
        continue
    end if
    Check whether HS is satisfiable
    if HS is satisfiable **then**
        Add the solutions returned for HS to S
    else
        $C \leftarrow C \cup \{\text{GBCoreIterativeRefinement}(HS)\}$ as in Section 5 of [2]
    end if
end for
    return S, C

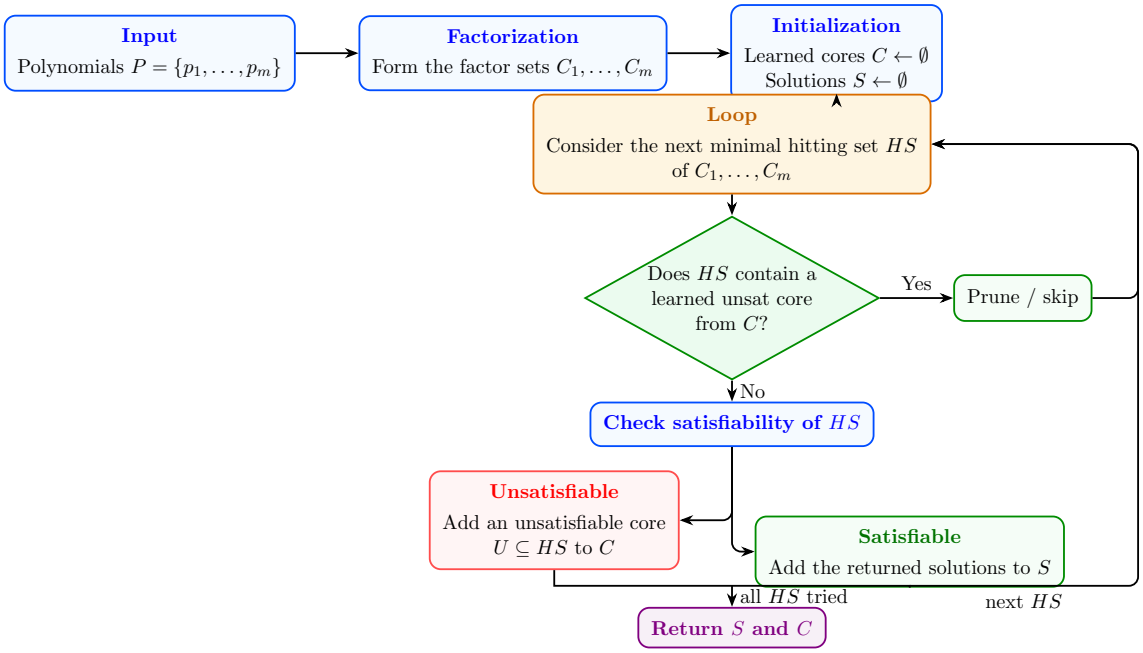


Figure 1: Flowchart of the branching procedure of Algorithm 1.

3.1 Iteration over minimal hitting sets

There are some additional details to consider so the iteration over the minimal hitting sets is efficient.

Order of iteration: The order of iteration over the hitting sets is from the simplest to the most complex to check for satisfiability. As a heuristic for this cost we use the rule more constraint factors, the more complex to check. This allows our method to learn early unsatisfiable cores that are easier to check and prune the search space.

Lazy iteration: The algorithm iterates over the hitting sets in a lazy manner, skipping hitting sets that contain a learned unsatisfiable core.

3.2 Theoretical analysis

Lemma 1 (Correctness of interreduction) *Suppose that one generator p_i is replaced by*

$$p_i' = p_i - \sum_{j \neq i} a_j p_j, \qquad a_j \in K[x_1, \ldots, x_n].$$

Then

$$\langle p_1, \ldots, p_i, \ldots, p_m \rangle = \langle p_1, \ldots, p_i', \ldots, p_m \rangle.$$

Any finite sequence of these reductions therefore preserves the generated ideal and its zero set.

Proof 1 *The polynomial p_i' belongs to the original ideal, while $p_i = p_i' + \sum_{j \neq i} a_j p_j$ belongs to the ideal generated by the updated system. Each generating set therefore generates the other. The same argument applies at every step of a sequential interreduction.*

A zero generator imposes no constraint and can be discarded. A nonzero constant makes the ideal the unit ideal, so the input system is unsatisfiable.

Lemma 2 (Sufficiency of minimal hitting sets) *Every hitting set T of the factor family contains a minimal hitting set $H \subseteq T$, with $V(T) \subseteq V(H)$. It is therefore enough to consider minimal hitting sets.*

Proof 2 *Because the factor sets are finite, repeatedly removing redundant factors from T eventually produces a minimal hitting set H. Since $H \subseteq T$, any point in $V(T)$ also lies in $V(H)$.*

Theorem 1 *The solution set of P is*

$$V(P) = \bigcup_{H \in \mathcal{H}} V(H).$$

Proof 3 *Take $x \in V(P)$. Since $p_i(x) = 0$, at least one factor in C_i vanishes at x for every i. The set of all factors that vanish at x is therefore a hitting set of $C_1, \ldots, C_m$. Because the factor sets are finite, this set contains a minimal hitting set H. Every factor in H vanishes at x, so $x \in V(H)$.*

For the reverse inclusion, take $x \in V(H)$ for some $H \in \mathcal{H}$. The set H intersects every C_i, so each p_i has a factor that vanishes at x. Hence $p_i(x) = 0$ for every i, and $x \in V(P)$.

The same decomposition can be stated in terms of ideals.

Theorem 2 (Radical decomposition) *The radical of the input ideal satisfies*

$$\sqrt{\langle P \rangle} = \bigcap_{H \in \mathcal{H}} \sqrt{\langle H \rangle}.$$

Proof 4 *By Theorem 1, $V(P) = \bigcup_{H \in \mathcal{H}} V(H)$. Taking vanishing ideals gives*

$$I(V(P)) = I\left(\bigcup_{H \in \mathcal{H}} V(H) \right) = \bigcap_{H \in \mathcal{H}} I(V(H)).$$

Hilbert's Nullstellensatz identifies $I(V(P))$ with $\sqrt{\langle P \rangle}$ and each $I(V(H))$ with $\sqrt{\langle H \rangle}$, which gives the stated equality.

Lemma 3 *Let U be an unsatisfiable core. If $U \subseteq H$, then $V(H) = \emptyset$.*

Proof 5 *Since every point in $V(H)$ must satisfy every polynomial in U, we have $V(H) \subseteq V(U)$. The core U is unsatisfiable, so $V(U) = \emptyset$ and therefore $V(H) = \emptyset$.*

Soundness does not require U to be minimal. A smaller core is useful only because more later branches may contain it.

For the following result, assume that the branch solver terminates on every unpruned branch H. If H is satisfiable, it returns exactly $V(H)$. If H is unsatisfiable, it returns a certified core $U \subseteq H$.

Theorem 3 *Algorithm 1 is sound and complete for P.*

Proof 6 *Let P' denote the system after the optional interreduction step. We check that each part of the algorithm preserves the solution set.*

Interreduction. *Lemma 1 gives $\langle P' \rangle = \langle P \rangle$, and hence $V(P') = V(P)$.*

Factorization and branch construction. *Theorem 1, applied to the factor sets of P', gives*

$$V(P') = \bigcup_{H \in \mathcal{H}} V(H).$$

Lemma 2 shows that no nonminimal hitting set can add a solution missing from this union. Thus the initial branch collection represents exactly $V(P')$.

Loop invariant. *Let R be the set of branches that have not yet been processed, and let S be the solution set accumulated so far. At the start of the loop,*

$$S \subseteq V(P') \qquad \text{and} \qquad V(P') \subseteq S \cup \bigcup_{H \in R} V(H).$$

The first inclusion is the soundness invariant. The second is the completeness invariant: every solution not yet in S remains in a pending branch. Both hold initially because $S = \emptyset$, $R = \mathcal{H}$, and the branch decomposition holds.

Core pruning. *Suppose the algorithm skips H because it contains a learned core U. Lemma 3 gives $V(H) = \emptyset$. Removing H from R therefore leaves the completeness invariant unchanged, so soundness is unchanged as well.*

Branch solving. *If H is satisfiable, the solver adds exactly $V(H)$ to S. Since $V(H) \subseteq V(P')$, the soundness invariant is preserved. Moving $V(H)$ from the pending union into S also preserves the completeness invariant. If H is unsatisfiable, then $V(H) = \emptyset$, so removing it from R preserves both invariants.*

Return. *When the loop ends, $R = \emptyset$. The completeness invariant gives $V(P') \subseteq S$, while the soundness invariant gives $S \subseteq V(P')$. Hence $S = V(P') = V(P)$.*

Note that the theoretical analysis does not require the field K to be algebraically closed. This is due to if there is no solution over the algebraic closure, then there is no solution over the field itself.

4 Branching with variable zeroing

There are methods !!!Add citations!!! in the literature of algorithmic algebraic geometry that assume the value of a variable is zero, and then solve the resulting system.

The following method takes the algorithm 1 and allows for the solver to time out. If the solver times out, the method chooses a variable, and tries to solve the system using algorithm 1 once assuming the variable is zero, and once assuming the variable is non-zero. The branching procedure is then applied recursively either until every branch is resolved, or until the computation resources are exhausted. In the experiments section 5, we show that recursive branching 2 utilizes the computation resources better than the original branching method 1.

Algorithm 2 Recursive branching with variable zeroing

Require: A polynomial system P, a set of variables X, and a timeout τ.

function ZeroingBranching(P, X, τ)
  $C \leftarrow \emptyset$ ▷ One global unsatisfiable-core set
  return SolveByZeroing(P, X, τ, C)
end function
function SolveByZeroing(P, X, τ, C)
  $(S, T) \leftarrow$ BranchingWithTimeout(P, τ, C) ▷ Algorithm 1; C is shared and updated
  $U \leftarrow \emptyset$ ▷ Branches left unresolved
  for all $Q \in T$ **do**
    if $X = \emptyset$ **then**
      $U \leftarrow U \cup \{Q\}$
      continue
    end if
    $x \leftarrow$ ChooseVariable(Q, X)
    $(S_0, U_0, C) \leftarrow$ SolveByZeroing($Q \cup \{x\}, X \setminus \{x\}, \tau, C$)
    $(S_1, U_1, C) \leftarrow$ SolveByZeroing($Q \cup \{1 - xz\}, X \setminus \{x\}, \tau, C$) ▷ z is fresh
    $S \leftarrow S \cup S_0 \cup S_1$
    $U \leftarrow U \cup U_0 \cup U_1$
  end for
  return (S, U, C)
end function

Solutions of a Q_1 branch are projected onto the original variables, so the fresh variables do not appear in the output.

The run on `systems/system.txt`, with a five-second timeout for each cover, produces the recursion tree in Figure 2. The variable chosen after a timeout is written inside the corresponding cover node. Its outgoing edges give the zero and nonzero cases.

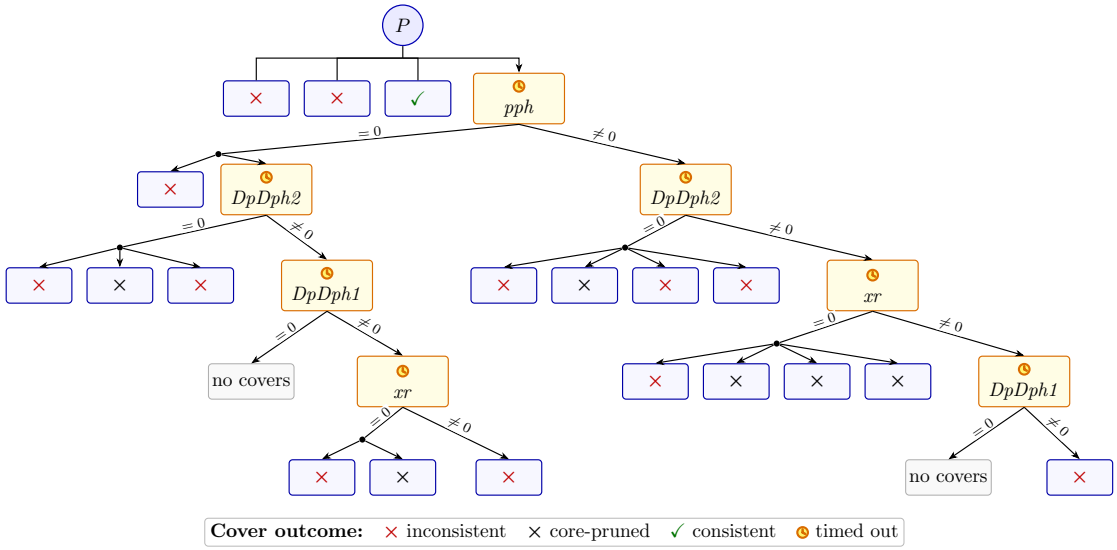


Figure 2: Recursion tree for `systems/system.txt`. Each cover is shown separately.

4.1 Choice of variable for splitting

The choice of variable for splitting effects the efficiency of the algorithm. Every split creates at least two new covers to check. If a variable barely occurs in the current cover, both recursive calls may repeat most of the same work. A variable that occurs in many generators is a more promising choice: setting it to zero removes all terms containing it at once, while in the nonzero branch the algorithm can use the $1 - xz$.

For each candidate variable x that is still available on the current branch, define

$$\begin{aligned} g_Q(x) &= \#\{q \in Q : x \text{ occurs in } q\}, \\ t_Q(x) &= \#\{(q, m) : q \in Q,\ m \text{ is a term of } q \deg_x(m) > 0\}, \\ e_Q(x) &= \sum_{q \in Q} \sum_{m \text{ a term of } q \deg_x(m) > 0} \deg_x(m). \end{aligned}$$

Its score is the triple

$$h_Q(x) = \big(g_Q(x), t_Q(x), e_Q(x)\big),$$

and scores are compared lexicographically.

4.2 Theoretical analysis of variable zeroing

Fix the variables Y of the input system, and let π_Y denote projection onto these variables. For a system Q at any node of the recursion, write

$$V_Y(Q) = \pi_Y(V(Q)).$$

Thus, π_Y removes the fresh inverse variables introduced on the path to Q.

Lemma 4 (Correctness of a zeroing split) *Let x be a variable of Q and let z be fresh. Then*

$$V_Y(Q) = V_Y(Q \cup \{x\}) \cup V_Y(Q \cup \{1 - xz\}).$$

Proof 7 *Take a solution of Q. If its value at x is zero, it is a solution of $Q \cup \{x\}$. Otherwise, extend it by setting $z = x^{-1}$. The extension satisfies $1 - xz = 0$, so its projection belongs to $V_Y(Q \cup \{1 - xz\})$. Conversely, a solution of either child system satisfies every polynomial in Q. Its projection therefore belongs to $V_Y(Q)$.*

We use the following contract for the branching routine with a timeout. If it returns (S, T) on P, then S contains the solutions obtained from resolved branches and T contains the branches on which the solver timed out. Hence

$$V_Y(P) = S \cup \bigcup_{Q \in T} V_Y(Q), \qquad S \subseteq V_Y(P). \tag{1}$$

This follows from Theorem 3: resolved satisfiable branches contribute their solution sets, certified unsatisfiable branches contribute nothing, and timed-out branches remain in T. Sharing the set of certified cores does not change this equality by Lemma 3.

Theorem 4 (Variable zeroing) *Suppose that every completed call to the branching routine with a timeout satisfies Equation* (1)*. If the recursive procedure returns (S, U, C) on input (P, X, τ, C), then*

$$V_Y(P) = S \cup \bigcup_{Q \in U} V_Y(Q), \qquad S \subseteq V_Y(P).$$

In particular, the solutions in S are sound. If $U = \emptyset$, they are also complete.

Proof 8 *We induct on $|X|$. The initial call to the branching routine gives Equation* (1)*. If $X = \emptyset$, the algorithm moves every branch in T to U, so the claimed equality is unchanged.*

Now suppose $X \neq \emptyset$. For each $Q \in T$, the algorithm chooses $x \in X$ and replaces Q by its zero and nonzero children. By Lemma 4, their projected solution sets have union $V_Y(Q)$. Each recursive call has one fewer available variable, so the induction hypothesis applies to both children. Replacing their solution sets by the corresponding solved and unresolved outputs preserves the union in Equation (1)*. It also preserves $S \subseteq V_Y(P)$, since every child solution comes from a branch of P. After all branches in T have been processed, the accumulated sets S and U satisfy the stated equalities.*

At a timed-out node, the outcomes of the two children determine the outcome of their parent. If both children are unsatisfiable, the parent is unsatisfiable. Otherwise, its resolved solution set is the union of the resolved solution sets of its children, while any branches that time out again remain in U. Thus an interrupted run still returns a sound partial solution set together with the frontier that contains every solution not yet found.

5 Experiments

5.1 Setup

We implemented Algorithm 1 in Rust 1.97.1 and used Symbolica 1.5 for polynomial arithmetic over the rationals. A branch can be checked by M4GB, F5, GVW, Symbolica's built-in Gröbner basis implementation, or Singular 4.4.1. The comparison also includes six direct whole-ideal Gröbner basis configurations, three decomposition methods from Singular, Macaulay2 1.26.06, and a numerical method from HomotopyContinuation.jl under Julia 1.12.6.

We ran the benchmark on an Apple M4 Pro with 14 CPU cores and 24 GB of memory. Every table entry is a single run. Every method received 60 seconds per system.

For branching, T is end-to-end wall time and F is the time from the start of the factor-cover search to its first satisfiable cover. Factorization and input processing are therefore included in T but not in F. The complete measurements are stored in the appendix.

5.2 Baseline systems

The baseline has 17 generated systems in six families. Signed-distance cycles and affine-factor hypergraphs test many inexpensive covers. The overlapping random multiples use seeded dense quadratic factors that recur across equations, with no factor shared by every equation. In the gated systems, a single selector multiplies every equation of a Cyclic, Katsura, or dense quadratic system. This produces one easy selector cover and one nonlinear residual cover.

Family	Tested sizes	Variables/equations; covers
Signed-distance cycle	$n = 8, 10, 12$	$8/8, 10/10, 12/12$; $8, 8, 32$
Overlapping random multiples	$(n, p, k) = (4, 6, 2), (6, 8, 3), (8, 10, 3)$	n/n; $4, 16, 28$
Positive affine hypergraph	$(n, m, k) = (8, 6, 4), (10, 8, 5), (12, 10, 6)$	n/m; $63, 239, 833$
Gated Cyclic	orders $5, 6$	$6/5, 7/6$; $2, 2$
Gated Katsura and dense quadratics	Katsura $6, 7$; dense $n = 5, 6$	$8/7, 9/8, 6/5, 7/6$; 2 each
Square affine hypergraph	$(n, k) = (8, 4), (10, 5)$	n/n; $31, 304$

Table 1: The generated systems used for the baseline benchmark. The overlapping random multiples use seed 31415; both affine families use seed 42.

The overlapping-random-multiple runs disable interreduction so that the generated factor-incidence pattern reaches the cover enumerator unchanged. All other branching runs use interreduction. Table 2 selects the fastest completed branching backend and the fastest completed method in each baseline class. A branching entry gives T/F in seconds; the other entries give total wall time. "Timeout" means that every tested method in that class timed out.

5.3 Physical Systems

6 Conclusion

References

[1] David A. Cox, John Little, and Donal O'Shea. *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*. Springer, Cham, 4 edition, 2015.

[2] Xiaojing Sun, Ioana Ilioaea, Priyank Kalla, and Florent Enescu. Finding unsatisfiable cores of a set of polynomials using the Gröbner basis algorithm. In *Principles and Practice of Constraint Programming*, volume 9892 of *Lecture Notes in Computer Science*, pages 859–875. Springer, 2016.

System	Best branching T/F	Best direct GB	Best decomposition
Signed distance, $n = 8$	Symb. 0.008/0.000	built-in 0.008	facstd 0.190
Signed distance, $n = 10$	Symb. 0.025/0.002	built-in 0.029	facstd 0.430
Signed distance, $n = 12$	Symb. 0.087/0.002	built-in 0.472	primdec 3.056
Overlapping $(4, 6, 2)$	F5 0.008/0.000	built-in 0.413	facstd 0.265
Overlapping $(6, 8, 3)$	F5 0.132/0.001	timeout	timeout
Overlapping $(8, 10, 3)$	Singular 2.931/0.010	timeout	timeout
Positive affine $(8, 6, 4)$	F5 0.008/0.000	timeout	facstd 0.368
Positive affine $(10, 8, 5)$	F5 0.020/0.000	timeout	facstd 1.836
Positive affine $(12, 10, 6)$	F5 0.067/0.000	timeout	facstd 53.318
Gated Cyclic-5	Symb. 0.009/0.000	built-in 0.008	facstd 0.181
Gated Cyclic-6	Singular 0.045/0.008	Singular 0.309	M2 2.620
Gated Katsura-6	Singular 0.049/0.008	GVW 0.226	facstd 0.233
Gated Katsura-7	Singular 0.283/0.008	GVW 3.176	facstd 0.539
Gated dense, $n = 5$	Singular 0.029/0.008	M4GB 0.171	facstd 0.198
Gated dense, $n = 6$	Singular 0.546/0.008	M4GB 4.736	facstd 0.755
Square affine $(8, 4)$	Singular 0.470/0.023	timeout	facstd 0.601
Square affine $(10, 5)$	Symb. 1.174/0.000	timeout	facstd 2.141

Table 2: Best completed result in each method class. Branching entries show wall time and first-solution latency; baseline entries show wall time. All times are in seconds.

A Appendix